

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

双指针算法

Two Pointers Technique

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia
2026 年 11 月 2 日

目录

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

1. 双指针概述
2. 对撞指针
3. 快慢指针
4. 滑动窗口
5. 总结



目录

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

1. 双指针概述
2. 对撞指针
3. 快慢指针
4. 滑动窗口
5. 总结

什么是双指针

双指针：在遍历数据时，使用两个指针（下标）协同工作，从而减少循环层数、降低时间复杂度的技巧。

核心优势

- 很多需要两重循环 $O(n^2)$ 的暴力题，双指针可以优化到 $O(n)$。
- 通过两个指针的移动，动态维护区间信息。

生活中的类比：两个人相向而行

- 一个从街头出发向右走，一个从街尾出发向左走。
- 两个人每次根据相遇点的情况，决定是谁往前走一步。
- 只要路线是单调的，两人最终一定会相遇。

双指针的本质：利用单调性，避免重复扫描已经确定不可能的区域

双指针的三大常见分类

- **对撞指针**：一左一右，相向而行。
  - 典型场景：有序数组的两数之和、反转字符串、验证回文串。
- **快慢指针**：同向而行，一个走得快，一个走得慢。
  - 典型场景：判断链表有环、找链表中点、原地删除重复项。
- **滑动窗口**：两个指针维护一个区间，右指针扩展，左指针收缩。
  - 典型场景：长度最小的子数组、无重复字符的最长子串。

不要死记硬背，理解指针移动的单调性是核心

目录

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. 双指针概述
2. 对撞指针
3. 快慢指针
4. 滑动窗口
5. 总结

例题一：两数之和 II

题目：给定一个已按照**升序排列**的整数数组 `numbers`，请你从数组中找出两个数满足相加之和等于目标数 `target`。假设每个输入只对应唯一的答案。

示例

- 输入：`numbers = [2, 7, 11, 15], target = 9`
- 输出：`[1, 2]`（2 和 7 相加为 9，下标从 1 开始）

两数之和 II：解法

核心思路：定义左右两个指针 `l` 和 `r`，分别指向数组的头和尾。计算当前和，根据与 `target` 的大小关系决定移动哪个指针。

关键步骤

- 第一步：初始化 `l = 0`，`r = n - 1`。
- 第二步：计算 `sum = numbers[l] + numbers[r]`。
- 第三步：如果 `sum == target`，直接返回下标。
- 第四步：如果 `sum > target`，说明右指针太大了，`r--`。
- 第五步：如果 `sum < target`，说明左指针太小了，`l++`。

为什么正确：因为数组有序，小了只能增大左指针，大了只能减小右指针

两数之和 II：代码


```cpp
vector<int> twoSum(vector<int>& numbers, int target) {
    int l = 0, r = numbers.size() - 1;
    while (l < r) {
        int sum = numbers[l] + numbers[r];
        if (sum == target) {
            return {l + 1, r + 1}; // 题目要求下标从 1 开始
        } else if (sum > target) {
            r--;                   // 和太大，右指针左移
        } else {
            l++;                   // 和太小，左指针右移
        }
    }
    return {};
}
```

目录

1 双指针概述

2 对撞指针

3 快慢指针

4 滑动窗口

5 总结

例题二：删除有序数组中的重复项

题目：给定一个升序排列的数组 `nums`，请你**原地**删除重复出现的元素，使每个元素只出现一次，返回删除后数组的新长度。

示例

- 输入：`nums = [1, 1, 2]`
- 输出：2，并且原数组前两个元素被修改为 1, 2。
- 输入：`nums = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]`
- 输出：5，修改后的数组前五个元素为 0, 1, 2, 3, 4。

删除有序数组中的重复项：解法

核心思路：使用快慢指针。慢指针 `slow` 指向当前不重复区间的最后一个位置，快指针 `fast` 负责往后探索新元素。

关键步骤

- 第一步：初始化 `slow = 0`，`fast = 1`。
- 第二步：比较 `nums[slow]` 和 `nums[fast]`。
- 第三步：如果相等，说明是重复项，`fast++` 跳过。
- 第四步：如果不等，说明遇到了新元素，将 `slow++`，并把新元素放到 `nums[slow]`。
- 第五步：无论是否交换，`fast++` 继续向前。

快指针探路，慢指针守阵地；遇到新东西，慢指针才动

删除有序数组中的重复项：代码


```cpp
int removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.size(); fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;                     // 慢指针前进一格 *
            nums[slow] = nums[fast];    // 覆盖旧值，保留新元素
        }
    }
    return slow + 1;                    // 返回新数组长度
}
```

目录

1 双指针概述

2 对撞指针

3 快慢指针

4 滑动窗口

5 总结



例题三：无重复字符的最长子串

题目：给定一个字符串 `s`，请你找出其中不含有重复字符的**最长子串**的长度。

示例

- 输入：`s = "abcabcbb"`
- 输出：3，因为无重复字符的最长子串是"abc"。
- 输入：`s = "bbbbb"`
- 输出：1，因为最长子串是"b"。

无重复字符的最长子串：解法

核心思路：用左右指针 `l` 和 `r` 维护一个无重复字符的滑动窗口，用数组记录每个字符最后一次出现的下标。

关键步骤

- 第一步：右指针 `r` 不断向右扩展，读取字符 `c`。
- 第二步：如果 `c` 曾经出现过，并且出现的位置在 `l` 之后，说明窗口内有重复了。
- 第三步：更新左指针 `l`，直接跳到 `c` 上次出现位置的下一个。
- 第四步：更新字符 `c` 的最新出现位置。
- 第五步：每次移动后，用当前窗口长度 `r - l + 1` 更新最大长度。

遇到重复的，左边界直接跳过去，不用一步步挪

无重复字符的最长子串：代码


```cpp
int lengthOfLongestSubstring(string s) {
    vector<int> last_pos(128, -1); // 记录 ASCII 字符最后出现的位置
    int ans = 0;
    int l = 0;

    for (int r = 0; r < s.size(); r++) {
        char c = s[r];
        // 如果字符出现过，并且位置在当前窗口内 *
        if (last_pos[c] >= l) {
            l = last_pos[c] + 1;   // 左指针直接跳过重复位置
        }
        last_pos[c] = r;           // 更新最新位置 *
        ans = max(ans, r - l + 1); // 更新答案
    }
    return ans;
}
```

目录

1 双指针概述

2 对撞指针

3 快慢指针

4 滑动窗口

5 总结



总结：双指针套路速查

类型	指针移动方式	典型应用
对撞指针	`l++`，`r--`	有序数组两数之和、回文串
快慢指针	`slow` 条件满足才走	原地去重、找中点、判环
滑动窗口	`r` 扩张，`l` 收缩	最长无重复子串、最小覆盖子串

核心优势

- 避免嵌套循环，把 $O(n^2)$ 的暴力解法优化到 $O(n)$。
- 两个指针都只朝一个方向移动，最多各自走 n 步。

总结：常见易错点

- **指针越界**：循环条件写 `l < r` 还是 `l <= r`，取决于是否允许重合。
- **忘记更新状态**：滑动窗口左指针移动后，要记得更新窗口内的统计状态。
- **死循环**：如果两个指针在某次循环中都没有移动，可能导致死循环。一定要保证每轮循环都有指针在动。
- **对撞指针的前提**：数据必须具有单调性（比如已排序），否则无法判断该移动哪个指针。

双指针题，先想清楚指针代表什么，再想清楚何时移动

附录：回文串验证（对撞指针）


```cpp
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        // 跳过非字母数字字符 *
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;

        if (tolower(s[l]) != tolower(s[r])) {
            return false;
        }
        l++;
        r--;
    }
    return true;
}
```